

Software Requirements Specification

Legal Document Analyser and Adviser

1. Introduction

1.1 Purpose

This document specifies the requirements for the **Legal Document Analyser and Adviser**, a web-based system built entirely on the Java platform. Users upload legal documents (contracts, agreements, notices, policies), receive plain-language summaries, identify key clauses and risks, and get general guidance on next steps. It is intended for developers, testers, project guides and evaluators.

1.2 Scope

The system accepts legal documents in common formats, extracts their text, analyses it using Natural Language Processing (NLP) and a Large Language Model (LLM), and presents structured results: summary, clause breakdown, risk highlights, glossary of legal terms and suggested actions. It also provides a question-and-answer adviser so users can ask about their document. The system provides informational assistance only and does not replace a qualified lawyer.

1.3 Definitions and Abbreviations

- **SRS:** Software Requirements Specification
- **NLP:** Natural Language Processing
- **LLM:** Large Language Model
- **OCR:** Optical Character Recognition
- **RAG:** Retrieval-Augmented Generation
- **API:** Application Programming Interface
- **JWT:** JSON Web Token
- **JDK:** Java Development Kit
- **JPA:** Jakarta Persistence API (ORM specification, implemented by Hibernate)
- **MVC:** Model-View-Controller
- **DTO:** Data Transfer Object

1.4 Intended Audience

Students, small business owners, tenants, freelancers and general citizens who must understand legal documents without legal training; legal interns and paralegals seeking a quick first-pass review.

2. Overall Description

2.1 Product Perspective

A standalone Java web application with a browser-based front end, a Spring Boot backend, an AI analysis engine and a relational database. It integrates with an external LLM service (called over HTTP from Java) and an OCR engine (Tesseract through the Tess4J library).

2.2 Product Functions (Summary)

Document upload, text extraction, summarisation, clause identification, risk detection, legal-term explanation, question-and-answer advice, report download and history management.

2.3 User Classes

- **General User:** uploads documents, reads analysis, asks questions.
- **Registered User:** all of the above plus saved history and report downloads.
- **Administrator:** manages users, monitors usage, updates prompts, rules and knowledge base.

2.4 Operating Environment

Modern web browsers (Chrome, Edge, Firefox, Safari) on desktop and mobile. The server runs on JDK 17 or later (JDK 21 LTS recommended) on a Linux-based cloud or local server; the application is platform independent because it runs on the JVM.

2.5 Design and Implementation Constraints

See Section 13 for detailed security, regulatory, and technical constraints.

3. System Features

ID	Feature	Description
F1	User Authentication	Registration, login, logout, password reset (Spring Security)
F2	Document Upload	Upload PDF, DOCX, TXT and scanned images (Spring MVC MultipartFile)
F3	Text Extraction	Parse text with PDFBox and Apache POI; OCR for scanned files with Tess4J
F4	Document Classification	Detect type: rental, employment, NDA, loan, sale, etc.
F5	Summarisation	Short and detailed plain-language summaries
F6	Clause Extraction	Identify parties, dates, obligations, payment, termination, jurisdiction
F7	Risk Detection	Flag unfair, ambiguous, missing or high-risk clauses with severity
F8	Legal Term Explainer	Simple definitions of jargon in context
F9	Legal Adviser Chat	Ask questions about the document and get grounded answers
F10	Suggestions	Recommended actions, questions to ask, clauses to negotiate
F11	Report Generation	Download analysis as PDF (OpenPDF or iText)
F12	History Management	View, search and delete past analyses
F13	Multilingual Support	English and Hindi input and output (optional extension)
F14	Admin Dashboard	User management, logs, analytics

4. Functional Requirements

ID	Requirement
FR1	The system shall allow users to register with name, email and password.
FR2	The system shall authenticate users and maintain secure sessions.
FR3	The system shall accept PDF, DOCX, TXT, JPG and PNG uploads up to a configurable size limit (default 10 MB, set through <code>spring.servlet.multipart.max-file-size</code>).
FR4	The system shall validate file type and size and reject unsupported or corrupt files with a clear message.
FR5	The system shall extract text from digital documents and apply OCR to scanned ones.
FR6	The system shall classify the document type and display it to the user.
FR7	The system shall generate a summary in simple language.
FR8	The system shall extract key clauses and key entities (parties, dates, amounts, duration, governing law).
FR9	The system shall detect risky, one-sided or missing clauses and assign Low, Medium or High severity with an explanation.
FR10	The system shall explain legal terms found in the document.
FR11	The system shall answer user questions using the uploaded document as the primary context.
FR12	The system shall provide suggested next steps and a disclaimer that output is not legal advice.
FR13	The system shall allow users to export results as a PDF report.
FR14	The system shall store analysis history for registered users and allow deletion at any time.
FR15	The system shall show processing progress and handle failures with retry options.
FR16	The system shall let administrators view users, activity logs and system statistics.
FR17	The system shall let users give feedback (helpful or not helpful) on each analysis.

5. Non-Functional Requirements

- **Performance:** A document of up to 20 pages shall be analysed within 60 seconds under normal load; page loads under 3 seconds. Long-running analysis runs asynchronously (Spring @Async or an ExecutorService thread pool) so the UI is never blocked.
- **Scalability:** Support at least 100 concurrent users through horizontal scaling of stateless Spring Boot instances behind a load balancer.
- **Reliability and Availability:** Target 99% uptime; graceful handling of LLM or OCR service failure using timeouts, retries and fallbacks (for example Resilience4j).
- **Usability:** Clean interface, plain language, responsive design, minimal steps from upload to result.
- **Accuracy:** Extracted clauses and summaries shall be traceable to source text; the system shall not invent clauses.
- **Maintainability:** Layered, modular code with documented APIs (OpenAPI/Swagger), configurable prompts and rules stored outside the code.
- **Portability:** Runs on Windows, Linux and macOS because the JVM is platform independent; also packaged as a Docker image.
- **Accessibility:** Follow WCAG 2.1 AA guidelines where practical.
- **Compatibility:** Works on current versions of major browsers (Chrome, Edge, Firefox, Safari).

6. User Requirements

- Users shall be able to upload a document and understand it without legal knowledge.

- Users shall be able to see which parts are risky and why.
- Users shall be able to ask follow-up questions in natural language.
- Users shall be able to save or download results.
- Users shall be assured that their documents remain private.
- Administrators shall be able to monitor and maintain the system without code changes.

7. System Requirements

7.1 Hardware (Server)

- **Processor:** 4 cores or higher
- **RAM:** 8 GB minimum (16 GB recommended); JVM heap configured through -Xms and -Xmx
- **Storage:** 50 GB SSD or more
- **Network:** stable broadband connection for LLM API access
- **GPU:** optional, needed only for hosting local models

7.2 Hardware (Client)

Any device with a modern browser, 2 GB RAM, internet connection.

7.3 Software

- **Server OS:** Ubuntu 22.04 or equivalent (Windows or macOS for development)
- **Runtime:** JDK 17 or later (JDK 21 LTS recommended)
- **Build tool:** Apache Maven (or Gradle)
- **Application server:** embedded Tomcat provided by Spring Boot
- **Client:** Chrome, Edge, Firefox or Safari (latest two versions)

8. External Interface Requirements

8.1 User Interface

Pages: Home, Login/Register, Upload, Analysis Result (tabs for Summary, Clauses, Risks, Terms, Chat), History, Profile and Admin Dashboard. Risk levels use colour and text labels. The layout is responsive. Pages are rendered with Thymeleaf templates and Bootstrap (a React front end calling the REST API is an acceptable alternative).

8.2 Hardware Interface

No special hardware; standard input devices, optional camera or scanner for document images.

8.3 Software Interfaces

- **LLM provider API:** For summarisation, extraction and chat, called through Spring WebClient or the Java 11+ HttpClient (or the LangChain4j library).
- **OCR engine:** Tesseract accessed through Tess4J.
- **Database connector:** JDBC with Spring Data JPA and Hibernate.
- **PDF generation library for reports:** OpenPDF or iText.
- **Optional email service:** Spring Mail (JavaMail) for verification and password reset.

8.4 Communication Interfaces

HTTPS with REST and JSON between client and server; TLS 1.2 or higher; SMTP for email notifications.

9. Database Requirements

9.1 Main Entities (JPA @Entity classes)

- **User:** user_id, name, email, password_hash, role, created_at
- **Document:** doc_id, user_id, file_name, file_type, storage_path, uploaded_at, doc_type
- **Analysis:** analysis_id, doc_id, summary, status, created_at
- **Clause:** clause_id, analysis_id, clause_type, text, page_ref
- **Risk:** risk_id, analysis_id, clause_id, severity, explanation, suggestion
- **ChatMessage:** msg_id, analysis_id, role, content, timestamp
- **Feedback:** feedback_id, analysis_id, rating, comment
- **Log:** log_id, user_id, action, timestamp

9.2 Relationships

One user has many documents (@OneToMany); one document has many analyses; one analysis has many clauses, risks and chat messages. Each relationship is mapped with JPA annotations and accessed through Spring Data repositories.

9.3 Data Requirements

Passwords stored only as salted hashes; uploaded files encrypted at rest; retention period configurable; user-initiated deletion removes the document, analysis and chat data (cascade delete). Regular automated backups. Schema changes are versioned with Flyway or Liquibase.

10. Software and Technology Requirements

Layer	Technology
Language and runtime	Java 17 or later (JDK 21 LTS recommended)
Backend framework	Spring Boot (Spring MVC for REST controllers, Spring Security for authentication and authorisation)
Frontend	Thymeleaf with HTML, CSS, JavaScript and Bootstrap; React optional
AI / NLP	LLM API via LangChain4j or HttpClient; Apache OpenNLP or Stanford CoreNLP for sentence splitting, tokenising and entity recognition; embeddings for semantic search
Vector store for RAG	PostgreSQL with pgvector, or Apache Lucene, or an in-memory store provided by LangChain4j
Document processing	Apache PDFBox (PDF), Apache POI (DOCX), Tess4J (Tesseract OCR)
Persistence	Spring Data JPA with Hibernate
Database	PostgreSQL or MySQL
Authentication	Spring Security with JWT (jjwt library), BCrypt password encoder
Report generation	OpenPDF or iText
Build tool	Maven or Gradle
Version control	Git and GitHub
Deployment	Executable JAR or Docker, cloud hosting or university server
Testing	JUnit 5, Mockito, Spring Boot Test, Postman, Selenium
API documentation	springdoc-openapi (Swagger UI)

11. System Architecture

The application follows a layered architecture:

- 1. **Presentation layer:** Thymeleaf views and REST controllers (@Controller, @RestController).
- 2. **Service layer:** Business logic (@Service) for authentication, document handling, analysis, chat and reporting.
- 3. **Repository layer:** Database access (@Repository, Spring Data JPA interfaces).
- 4. **AI engine module:** Classes that call the LLM, run NLP steps and perform retrieval.
- 5. **Database:** Relational store for users, documents, analyses and history.

11.1 Suggested Package Structure

Package	Contents
config	SecurityConfig, WebClientConfig, AsyncConfig, OpenApiConfig
controller	AuthController, DocumentController, AnalysisController, ChatController, AdminController
service	UserService, DocumentService, TextExtractionService, ClassificationService, AnalysisService, RiskService, ChatService, ReportService
ai	LlmClient, PromptBuilder, ChunkingService, EmbeddingService, VectorStoreService, PromptInjectionGuard
repository	UserRepository, DocumentRepository, AnalysisRepository, ClauseRepository, RiskRepository, ChatMessageRepository
model	JPA entities and enums (Role, Severity, DocType, AnalysisStatus)
dto	Request and response objects
exception	Custom exceptions and a @ControllerAdvice global handler
security	JwtUtil, JwtAuthFilter, UserDetailsServiceImpl

11.2 Java Concepts Demonstrated

The project is designed to exercise core and advanced Java topics: object-oriented design (inheritance, interfaces, polymorphism, for example a DocumentParser interface with PdfParser, DocxParser, TxtParser and ImageParser implementations), collections and generics, streams and lambdas, exception handling with custom exceptions, multithreading with ExecutorService and CompletableFuture, file I/O, JDBC and JPA, and annotations.

12. System Workflow

- 1. The user registers or logs in.
- 2. The user uploads a legal document.
- 3. The system validates the file and stores it securely.
- 4. Text is extracted (PDFBox, POI, or Tess4J OCR if the file is scanned).
- 5. Text is cleaned and split into sections or chunks.
- 6. The document type is classified.
- 7. The AI engine generates the summary, extracts clauses and entities, and detects risks (executed asynchronously).
- 8. Results are saved to the database and shown on the result page.
- 9. The user asks questions; the system retrieves relevant chunks and generates grounded answers.
- 10. The user downloads a report, gives feedback or deletes the document.

Data flow: User → Web UI → Spring Controller → Service Layer → Document Processor → AI Engine (with vector store) → Repository → Database → Web UI → User.

13. Security Requirements

- All traffic shall be encrypted with HTTPS.
- Passwords shall be hashed with Spring Security's BCryptPasswordEncoder (or Argon2); sessions use JWT with expiry.
- Role-based access control (ROLE_USER, ROLE_ADMIN) separates users and administrators using Spring Security method and URL security.
- Users shall access only their own documents and analyses.
- Uploaded files shall be scanned and validated (content type, extension and size) to prevent malicious uploads.
- Input validation (Jakarta Bean Validation), output escaping and JPA parameterised queries shall prevent injection and XSS attacks; CSRF protection is enabled for browser forms.
- Prompt-injection safeguards shall prevent document content from overriding system instructions.
- API keys shall be held in environment variables or external configuration, never in source code.
- Rate limiting shall protect against abuse (for example Bucket4j).
- Documents sent to third-party AI services shall be minimised, and users informed of this.
- Audit logs shall record sensitive actions.
- The system shall comply with applicable data protection law, including India's Digital Personal Data Protection Act, 2023.

14. Constraints

- The system gives informational output only and is not a substitute for a licensed lawyer.
- Accuracy depends on text quality, especially for poor scans and handwriting.
- Dependence on third-party LLM availability, cost and rate limits.
- Context-length limits restrict very long documents; chunking is required.
- Legal interpretation varies by jurisdiction; initial coverage targets Indian law.
- Java NLP libraries are less extensive than Python's, so most language understanding is delegated to the LLM, with OpenNLP used for lightweight pre-processing.
- Project timeline, budget and team size limit scope.

15. Assumptions

- Users have internet access and a modern browser.
- Uploaded documents are mostly in English; Hindi is optional.
- Documents are legible and not password-protected.
- The LLM and OCR services remain available and affordable.
- Users consent to automated analysis of their documents.
- Legal reference data used by the system is reasonably current.
- JDK 17 or later and Maven are installed in the development and deployment environments.

16. Future Scope

- Support for more regional languages and voice input
- Contract comparison and version difference detection
- Auto-drafting of legal documents and notices from templates
- Integration with Indian legal databases (acts, sections, case law)
- Citation of relevant statutes and judgments

- Lawyer marketplace for expert consultation
- E-signature and document tracking
- Mobile applications for Android (Java/Kotlin) and iOS
- Fine-tuned domain-specific legal models
- Deadline and renewal reminders extracted from contracts
- Offline or on-premise deployment for privacy-sensitive users

17. Conclusion

The Legal Document Analyser and Adviser aims to make legal documents understandable to ordinary people by combining document processing, NLP and LLM technology on a robust Java and Spring Boot foundation. By summarising content, identifying clauses, highlighting risks and answering questions in plain language, the system reduces dependence on costly manual review and helps users make informed decisions. This SRS defines the functional, non-functional, security and technical requirements for building a reliable, secure and user-friendly solution, and provides a base for future enhancements.